Computational Linguistics 1 — IIIT Hyderabad

Code-mixed Text Analysis

Ram Kowshik & Aayush Swami

IIIT Hyderabad — 30 March 2025

1 What is Code-Mixing

Think about how people actually talk in a city like Hyderabad or Mumbai. Someone will start a sentence in Telugu or Hindi and finish it in English without even thinking about it. That is code-mixing. It is not a mistake or a bad habit — it is just how multilingual people naturally speak and write.

More formally, code-mixing happens when a speaker uses words or phrases from two or more languages in the same sentence. Sridhar and Sridhar (1980) describe it as moving from the words of one language to the words of another within a single sentence. It is slightly different from code-switching, where a person changes languages between sentences rather than inside one. But in practice, people use both terms to mean the same thing, and in this project we use “code-mixed” to cover both situations (Wikipedia, 2024).

One thing worth clearing up is that code-mixing does not mean someone is confused or bad at languages. It is actually the opposite. Studies show that people who mix languages tend to be confident in both of them. They do it for different reasons: sometimes a word in one language just fits better, sometimes the other language does not have a good translation for what they want to say, and sometimes it is just a way of connecting with the people around them (Myers-Scotton, 1997).

In India, this is not a niche thing — it is everywhere. India has 22 official languages listed in the Eighth Schedule of the Constitution, and over 121 mother tongues were recorded in the 2011 census alone. English has become a common working language across professional and city life, used alongside Hindi, Telugu, Tamil, Kannada, Bengali, and many others (Wikipedia: Languages of India, 2026; Association for Asian Studies, 2023). Out of this, varieties like Hinglish — Hindi mixed with English — have grown naturally, and you can see them all over social media, YouTube comments, and WhatsApp chats every day.

A message like “yaar I’m so thak gaya from this semester” is completely normal to anyone who grew up in this environment. But for most NLP software, it is basically unreadable. Fixing that is what this project is about.

2 Relevance to Computational Linguistics

NLP tools have gotten very good over the last few years. They can translate languages, answer questions, summarise documents, and do a lot more. But almost all of that progress happened using clean, formal, single-language text — the kind you find in newspapers or textbooks. Social media is a completely different situation.

When you try to use standard NLP tools on code-mixed text, they struggle badly. A tokeniser built for English has no idea what to do with a Hindi word written in English letters. A sentiment model trained on English tweets will completely miss the emotion in a sentence where most of the feeling is expressed in another language. Even something as basic as detecting what language a word is in becomes hard, because the same Roman script is being used for multiple languages at once. Srivastava and Singh (2021) found that code-mixing causes “non-standard vocabulary, casual syntax, and phonetic spelling” that breaks standard NLP pipelines.

This is not just a technical inconvenience. It has real consequences. When a hate speech detector misses abuse written in Hinglish, real people are affected. When a public health tool cannot read what people in regional communities are saying about a disease outbreak, important information gets lost. Srivastava and Singh (2021) document how these failures show up across areas like crisis management, healthcare, fake news detection, and more.

The first step toward fixing any of this is normalisation — which is basically cleaning and standardising the raw text so that tools can actually process it. For code-mixed text, this is harder than usual. There is no standard spelling system for Hindi or Telugu written in English letters, so the same word might appear ten different ways. Language boundaries have to be figured out word by word, not at the document level (Das & Gambäck, 2014). Emoji and slang need to be handled too. These are exactly the stages our pipeline is designed to deal with.

There is also a bigger picture here. Code-mixing is not unique to India — you see it in Spanglish in the US, in Arabic-French mixing across North Africa, in Taglish in the Philippines. A huge portion of the world’s internet users communicate this way. A field that cannot handle code-mixed text is missing most of the world. That is why this kind of work matters, and why it keeps growing as a research area in computational linguistics.

References

Association for Asian Studies. (2023). Multilingualism in India. Education About Asia. Retrieved from https://www.asianstudies.org/publications/eaa/archives/multilingualism-in-india/

Das, A., & Gambäck, B. (2014). Identifying languages at the word level in code-mixed Indian social media text. Proceedings of the 11th International Conference on Natural Language Processing (pp. 378–387). NLP Association of India.

Khurana, D., Koli, A., Khatter, K., & Singh, S. (2022). Natural language processing: State of the art, current trends and challenges. PMC – National Library of Medicine. https://pmc.ncbi.nlm.nih.gov/articles/PMC9281254/

Myers-Scotton, C. (1997). Code-switching. In F. Coulmas (Ed.), The Handbook of Sociolinguistics. Blackwell, Oxford.

Poplack, S. (1980). Sometimes I'll start a sentence in Spanish y termino en Español: Toward a typology of code-switching. Linguistics, 18(7–8), 581–618.

Sridhar, S. N., & Sridhar, K. K. (1980). The syntax and psycholinguistics of bilingual code mixing. Canadian Journal of Psychology, 34(4), 407–416. (As cited in Wikipedia: Code-mixing, 2024.)

Srivastava, V., & Singh, M. (2021). Challenges and considerations with code-mixed NLP for multilingual societies. arXiv:2106.07823. https://www.researchgate.net/publication/352432102

Wikipedia. (2024). Code-mixing. Retrieved from https://en.wikipedia.org/wiki/Code-mixing

Wikipedia. (2026). Languages of India. Retrieved from https://en.wikipedia.org/wiki/Languages_of_India

---

NOTE:

There are some changes in the order of the contents from henceforth because of the relativity of concepts. The tool used, and the proposed approach, are furthermore detailed in the readme file present in the zip file.

Abstract

Code-mixing — the practice of alternating between two or more languages within a single

sentence — is extremely common in Indian social media text. This paper presents a rule-based and statistical NLP pipeline for **Tenglish** (Telugu-English code-mixed text), adapted from an existing Hinglish (Hindi-English) system. The pipeline performs two core tasks on Romanized Telugu-English social media text: **word-level Language Identification (LID)** and **Part-of-Speech (POS) tagging** using a Hidden Markov Model (HMM) with Viterbi decoding. The system was trained on 23,560 annotated tokens and evaluated on 5,892 tokens sourced from Telugu-English social media posts, achieving an overall POS accuracy of **65.78%** and LID accuracy of **55.33%**.

6.Dataset

6.1Source

The dataset was sourced from the **ICON 2016 Shared Task on POS Tagging for Code-Mixed Indian Social Media Text**, which provided annotated Telugu-English Facebook comment data. The data was accessed via the publicly available repository: `SunilGundapu/Word-Level-Language-Identification-in-English-Telugu-Code-Mixed-Data`

6.1Format and Preprocessing

The original data was in a 3-column space-separated format:

```
token   POS_tag   LID_label
```

A preprocessing script converted this into the project's required CSV format (`token,label`) and mapped the original tagset to the project's universal tagset. Blank rows were preserved as sentence boundaries.

1.1 Statistics

Split	Tokens	Sentences
Training	23,560	1,584
Test	5,892	392
Total	**29,452**	**1,976**

Training set LID distribution:

Language	Count	%
English (EN)	11,246	47.7%
Telugu (TE)	6,493	27.6%
Universal (UNI)	5,821	24.7%

Training set POS distribution (top tags):

Tag	Count	Description
N	8,695	Noun
X	4,423	Unclassified/Foreign
PUNC	3,123	Punctuation
V	2,335	Verb
ADJ	1,146	Adjective
ADP	1,010	Adposition/Postposition
PRON	928	Pronoun

3.Challenges in Telugu-English Code-Mixing

3.1 No Standard Romanization

Telugu has no official Roman script. Each writer spells the same word differently:

- *vaadu* / *vadu* / *vaaduu* / *wadu* — all mean “he” (Telugu:)
- *chestunnav* / *chestunav* / *cestunav* — “you are doing”

This means a token not seen during training is very likely a known word in a different spelling. **Addressed by:** phonetic hashing and the transliteration fallback dictionary.

3.2 Ambiguous Short Words

Short tokens appear in both languages with different meanings:

- *"lo"* — Telugu postposition ("in/inside") AND English slang ("lo and behold")
- *"em"* — Telugu question word ("what?") AND English abbreviation
- *"ok"* / *"okay"* — claimed by both indicator sets

Addressed by: context smoothing; whichever language dominates the surrounding tokens wins.

3.3 Proper Nouns (PROPN)

Names like *Prabhas, Garuda, Bahubali, Hyderabad* are neither purely Telugu nor English — they are named entities. The LID system often misclassifies them as TE (since most unknowns default to Telugu).

Addressed by: marking them as `UNI` in the ground truth, and adding a named-entity detection layer as a future improvement.

3.4 Social Media Noise

The data is drawn from Facebook comments and Twitter, containing:

- Emoticons: `:P` , `\m/`
- Hashtags: `#Garuda` , `#SSR`
- URLs: `pic.twitter.com/xyz`
- Repeated punctuation: `....` , `???`
- Mixed script noise: occasional native script characters in a Romanized sentence

Addressed by: the `_is_universal()` hard rule captures most of these as UNI before any language detection runs.

3.5 Telugu Postpositions Attached to English Words

Telugu postpositions are often appended directly to English words:

- *"officelo"* (office + lo = "in the office")
- *"meetingki"* (meeting + ki = "to the meeting")

This makes the fused token look like a Telugu word to the LID system. **Future fix:** a morphological segmenter to split such tokens before classification.

3.6 Class Imbalance

The training set has 8,695 Nouns vs. only 141 Interjections. The HMM is biased toward high-frequency tags like N and X, leading to lower recall for minority classes like PRON and CONJ.

---

4. How Errors Were Countered

Error Type	Example	Cause	Counter-measure Applied
Spelling variant not recognized	`"chpdu"` misclassified	Not in indicator list	Phonetic hash fallback
Known word in different spelling	`"vaaduu"` unknown	Extra vowel	Vowel-stripping in hash
Ambiguous short word	`"lo"` tagged EN when Telugu context	Both lists contain it	Context smoothing pass
Rare punctuation not tagged PUNC	`">"` , `"100%"` given wrong tag	Not in PUNC set	Hard override for punctuation tokens
Proper nouns default to TE	`"Prabhas"` TE instead of UNI	Unknown default TE	Default changed to respect `univ` label in data
Social media symbols as tokens	`"\m/"` , `":P"`	Treated as unknown words	`_is_universal()` catches non-alpha tokens first
Low recall on PRON	`"nuvvu"` tagged N	Emission probability lower than N	Expanding `te_indicators.json` with more pronouns

---

4. Pipeline Code

Full End-to-End Usage

```
import re
from approach2.statistical_models.code_switch_labeler import CodeswitchSegmenter
from approach2.statistical_models.hmm_pos_tagger import HMMPosTagger

def run_tenglish_pipeline(text: str):
    # Step 1: Initialize models
    segmenter = CodeswitchSegmenter()  # loads te/en indicator sets
    tagger    = HMMPosTagger()
    tagger.load()                      # loads pre-trained HMM weights

    # Step 2: Tokenize
    tokens = re.findall(r"[\w']+|[.,!?;:—…]", text)

    # Step 3: Language Identification
    lid_labels = segmenter.segment(tokens)       # ['TE','TE','EN','UNI',...]

    # Step 4: POS Tagging
    pos_results = tagger.viterbi_decode(tokens) # [(token, tag), ...]

    # Step 5: Print results
    print(f"{'Token':<20} | {'LID':<5} | {'POS'}")
    print("-" * 40)
    for i in range(len(tokens)):
        print(f"{tokens[i]:<20} | {lid_labels[i]:<5} | {pos_results[i][1]}")

# Example run
run_tenglish_pipeline(
    "Nuvvu ikkade undi, but I thought you were going to the market."
)
```

Sample Output:

```
Token               | LID   | POS
----------------------------------
Nuvvu               | TE    | X
ikkade              | TE    | N
undi                | TE    | N
,                   | UNI   | PUNC
but                 | EN    | CONJ
```

```
I          | EN  | PRON
thought      | EN  | V
you        | EN  | PRON
were       | EN  | V
going      | EN  | V
to         | TE  | ADP
the        | EN  | DET
market       | EN  | N
.          | UNI  | PUNC
```

Training the Model

```
# Train on your annotated CSV (token,label format)
tagger = HMMPosTagger()
tagger.train("data/raw/train_pos.csv")
# Saves: hmm_emissions.json, hmm_transitions.json,
#        hmm_phonetic_emissions.json, hmm_unigrams.json
```

Evaluating Accuracy

```
# POS evaluation
python src/approach2/tools/verify_hmm_accuracy.py --pos data/raw/test_pos.csv

# LID evaluation
python src/approach2/tools/verify_hmm_accuracy.py --lid data/raw/test_lid.csv
```

7.System Description: How the Code Works

7.1Overall Pipeline

Raw Tenglish Text

Tokenizer regex: [\w']+ | punctuation

CodeswitchSegmenter LID label per token (TE / EN / UNI)

1. Hard rule: non-alpha UNI

```
    2. Fallback dict        transliteration_variants.json   lookup
    3. Indicator  sets       te_indicators.json,  en_indicators.json
    4. Phonetic hash        handles spelling variants
    5. Context smoothing     neighbour-based  resolution  for  unknowns
    6. Default              unknown      TE




     HMMPosTagger              POS tag per token

    Emission:  P(w|tag)      + phonetic hash fallback
    Transition: P(t|t-1)     bigram transitions
    Decoding:  Viterbi      finds best tag sequence



  Output: Token | LID | POS   (per token)
```

7.1Tokenizer

Input text is split using a regular expression:

```
tokens = re.findall(r"[\w']+|[.,!?;:—…]", text)
```

This keeps contractions like *"don't"* intact and separates punctuation as individual tokens.

7.3Language Identification — `CodeswitchSegmenter`

Each token is classified as TE, EN, or UNI through a cascade of rules:

i. **Hard rule:** Tokens that are purely punctuation, digits, URLs, hashtags (`#`), or mentions (`@`) are labelled UNI immediately.

ii. **Transliteration fallback dictionary** (`transliteration_variants.json`): Maps common informal Telugu spelling variants to a canonical form before lookup (e.g., `"chestunna"` `"chestunnaa"`).

iii. **Indicator set membership:** The token (after normalization) is checked against hand-crafted Telugu and English indicator word lists.

    - `te_indicators.json` — 161 Romanized Telugu words (pronouns, postpositions, verbs, particles)
    - `en_indicators.json` — 235 English words (function words, common content words, social-media slang)

iv. **Phonetic hash fallback:** For unseen spellings, a Telugu-aware phonetic hash function collapses spelling variants to the same key (e.g., `"vaadu"`, `"vadu"`, `"vaaduu"` same hash). The hash is compared against pre-built hash sets of the indicator lists.

v. **Context smoothing:** Unknown tokens surrounded by same-language tokens inherit that language label.

vi. **Default:** Any remaining unknown token defaults to **TE** (most unknowns in Tenglish social media are Telugu words).

7.4 POS Tagger — `HMMPosTagger`

A first-order Hidden Markov Model is trained on the annotated CSV:

- **Emission probabilities:** `P(word | tag)` — how likely a word is given a tag
- **Phonetic emission fallback:** For unseen words, falls back to `P(phonetic_hash | tag)` to handle spelling variants
- **Transition probabilities:** `P(tag_t | tag_{t-1})` — bigram tag transitions including a `START` state
- **Viterbi decoding:** Finds the most probable tag sequence for the full token list

Smoothing: A floor value of `1e-10` prevents zero-probability paths.

Hard overrides:

- Punctuation tokens always receive the `PUNC` tag regardless of Viterbi output.

7.4 POS Tagset

Tag	Description	Tenglish Example
N	Noun	*illu, office, time*
V	Verb	*chestunnav, went, undi*
ADJ	Adjective	*chaala, busy, good*
ADV	Adverb	*ippudu, later, already*

PRON	Pronoun	*nuvvu, I, vaadu*
DET	Determiner	*the, a, oka*
ADP	Adposition	*lo, ki, tho, nunchi*
CONJ	Conjunction	*kani, but, and*
PART	Particle	*ani, ante, ayithe*
PROPN	Proper Noun	*Hyderabad, Ravi*
NUM	Number	*3, rendu, ten*
PUNC	Punctuation	*. , ! ?*
INTJ	Interjection	*ayyo, arre, abba*
X	Foreign/unclassified	*lol, omg*

7.4Phonetic Hashing — `get_phonetic_hash()`

Telugu Romanization is highly inconsistent in informal text. The phonetic hasher handles:

- Repeated characters: `aa a` , `ee e` , `oo o` , `dd d` , `tt t`
- Telugu-specific clusters: `ch C` , `sh S` , `ph F` , `bh B` , `kh K` , `th T` , `dh D`
- Near-identical sounds: `v/w W` , `s/z S` , `k/c/q K`
- Vowel removal from non-initial positions (consonant skeleton)

Example: `"cheppaadu" "CHPD"` , `"cheppadu" "CHPD"` , `"chepadu" "CHPD"` — all map to the same hash, allowing the model to generalize across spelling variants.

---

8. Accuracy Analysis

b. POS Tagging Results

Overall POS Accuracy: 65.78% (3,876 / 5,892 tokens correct)

Tag	Precision	Recall	F1-Score	Support
ADJ	0.786	0.713	**0.748**	345
ADP	0.660	0.714	0.686	234

ADV	0.752	0.626	0.683	131
CONJ	**0.955**	0.824	**0.884**	51
DET	0.762	**0.826**	0.793	132
INTJ	**0.935**	**0.853**	**0.892**	34
N	0.627	0.766	0.690	2,179
PART	0.700	0.632	0.664	144
PRON	0.567	0.555	0.561	191
PUNC	**0.995**	0.492	0.658	767
V	0.635	0.593	0.613	562
X	0.560	0.556	0.558	1,122

Key observations:

- **CONJ (0.884 F1)** and **INTJ (0.892 F1)** perform best — these tags have strong, unambiguous indicator words (*but, and* / *ayyo, arre*)
- **PUNC (0.995 precision)** — when the model predicts PUNC it is almost always right, but recall is low (0.492) because the HMM sometimes assigns other tags to punctuation-like tokens such as `">"` , `"100%"` , `".."`
- **PRON (0.561 F1)** and **X (0.558 F1)** are the weakest — Telugu pronouns have many spelling variants and the X category is a catch-all for noisy tokens
- **N dominates** (2,179 tokens = 37% of test set) — the model is biased toward predicting N for unknown words, inflating N recall at the cost of other tags

c. Language Identification Results

Overall LID Accuracy: 55.33% (3,260 / 5,892 tokens correct)

Language	Precision	Recall	F1-Score	Support
EN	0.752	0.406	**0.527**	2,884
TE	0.439	**0.874**	**0.584**	1,625
UNI	0.609	0.484	0.539	1,383

Key observations:

- **TE recall is very high (0.874)** — the system correctly identifies most Telugu tokens, but at the cost of misclassifying many English and UNI tokens as Telugu (low TE precision: 0.439)
- **EN precision is decent (0.752)** but recall is low (0.406) — many English words not in the indicator list default to TE
- **The main bottleneck is the small indicator list size** — only 161 TE + 235 EN indicators. Many common English words (e.g., *cinema, concept, standing*) are not in the list and default to TE
- **Proper nouns are a key failure mode** — names like *Prabhas, Garuda, Bahubali* are labelled UNI in the ground truth but default to TE in predictions

d. Error Analysis — Top Failure Modes

Failure Mode	Frequency	Example	Fix
English word not in indicator list defaults to TE	Very High	*cinema TE, concept TE*	Expand `en_indicators.json`
Proper noun defaults to TE instead of UNI	High	*Prabhas TE, Garuda TE*	Add named-entity detection
Punctuation tokens with non-standard form	Medium	`"100%"`, `">"` tagged N	Extend PUNC hard-rule set
Telugu pronoun tagged N or X	Medium	*nuvvu X, naaku X*	Add more TE pronouns to indicators
Ambiguous word context wins wrong language	Low	*lo* EN when context is TE	Improve context window size

e. Comparison — Hinglish vs. Tenglish

Metric	Hinglish (original)	Tenglish (this work)
Training tokens	~similar	23,560
POS accuracy	Higher (established tagset)	**65.78%**
LID accuracy	Higher (larger indicator lists)	**55.33%**

Main challenge	Hindi script variants	Telugu Romanization + no standard spelling

Tenglish is inherently harder than Hinglish for LID because:

i. Hindi has a larger NLP community and more pre-built resources
ii. Telugu Romanization is more variable (retroflex consonants, long vowels)
iii. The Telugu indicator list required manual construction from scratch

---

9. Conclusion

This paper presented a complete **Tenglish NLP pipeline** for Language Identification and POS Tagging of Telugu-English code-mixed social media text. The system was adapted from an existing Hinglish pipeline and achieves:

- **65.78% POS accuracy** — competitive for an HMM-only system on noisy social media text
- **55.33% LID accuracy** — limited by indicator list coverage, with clear paths to improvement

The primary finding is that **the HMM architecture transfers well from Hinglish to Tenglish**, but the language-specific resources (indicator wordlists, phonetic rules) require careful Telugu-specific tuning. Expanding the Telugu indicator list and adding named-entity detection are the two highest-impact improvements for future work.

Future Work

1. **Expand indicator lists** — add 500+ common Telugu words to `te_indicators.json`
2. **Named entity detection** — pre-label proper nouns as UNI before LID
3. **CRF-based LID** — replace rule-based segmenter with a Conditional Random Field (achieves F1 ~0.91 on similar data per prior work)
4. **Transformer-based POS** — fine-tune MuRIL or IndicBERT on Tenglish data
5. **Morphological segmentation** — split fused tokens like *"officelo"* → *"office"* + *"lo"* before tagging

---

References

1. Das, A. et al. (2016). *ICON 2016 Shared Task: POS Tagging for Code-Mixed Indian Social Media Text*. ICON 2016, IIT (BHU) Varanasi.

2. Gundapu, S. (2018). *Word-Level Language Identification in English-Telugu Code-Mixed Data*. GitHub Repository.
3. Kunchukuttan, A. et al. (2020). *AI4Bharat IndicNLP Catalog*. https://github.com/AI4Bharat/indicnlp_catalog
4. Solorio, T. & Liu, Y. (2008). *Learning to Predict Code-Switching Points*. EMNLP 2008.
5. Vyas, Y. et al. (2014). *POS Tagging of English-Hindi Code-Mixed Social Media Content*. EMNLP 2014.

Word count: ~2,400 words | Pipeline tested on: Telugu-English Facebook/Twitter comments (ICON 2016)